Università degli Studi di Milano
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
GIOVANNI DEGLI ANTONI



CORSO DI LAUREA TRIENNALE IN
INFORMATICA

OTTIMIZZAZIONE DELL'ASSEMBLATORE GENOMICO FLYE ATTRAVERSO PARALLELIZZAZIONE SU GPU

Relatore: Prof. Alberto Stabile
Correlatore: Dr. Giorgio Chiurato

Elaborato Finale di:
Paolo Giorgio Della Torre
Matr. Nr. 987609

Anno Accademico 2025-2026

Abstract

L'assemblaggio genomico è il processo di ricostruzione del genoma completo di un organismo a partire da *read*, sequenze di caratteri che rappresentano letture di frammenti di DNA. Un assemblatore è un software che implementa questo processo.

Il lavoro di questa tesi si basa su Flye, un assemblatore che elabora *read* lunghe centinaia di migliaia di caratteri, dette *long read*, le quali presentano un tasso di errore fino al 13%.

Flye, per assemblare il genoma, esegue una serie di passaggi partendo dall'insieme di *read* in input:

- unisce le *read* in base alle sovrapposizioni tra le loro estremità in modo arbitrario, e forma delle catene chiamate *disjointig*;
- costruisce il grafo delle ripetizioni allineando i *disjointig* con sé stessi per individuare le regioni ripetute del genoma;
- allinea le *read* sul grafo per trovare i percorsi che fa il genoma su di esso;
- estrae dal grafo i *contig*, ovvero i percorsi nel grafo che rappresentano tratti di genoma contigui e che insieme formano la bozza del genoma.

La bozza del genoma così costruito però mantiene gli errori presenti nelle *read* di input, ed è per questo che occorre applicare un ultimo passaggio, detto *polishing*, per correggerli.

Il *polishing* divide la bozza del genoma in numerosi frammenti chiamati bolle. Una bolla è una struttura contenente la sequenza del frammento di genoma, o sequenza *candidato*, e pezzi *read* appartenenti a quella zona del genoma, chiamati *rami*.

Per ogni bolla viene calcolata la sequenza candidato statisticamente più probabile in base ai *rami* a disposizione, esprimendo questa probabilità con uno *score* intero calcolato con l'algoritmo di Needleman–Wunsch.

Il problema principale dell'assemblaggio è l'elevato costo computazionale su grossi genomi. Questo lavoro di tesi punta quindi alla riduzione dei tempi di calcolo di Flye facendo un *porting* su GPU della costosa fase di correzione delle bolle del *polishing*, in modo da accelerarne i tempi di esecuzione.

Il *porting*, testato su uno stesso genoma, al variare del numero di *thread* CPU ha ottenuto dei fattori di *speedup* rispetto alla versione originale di Flye compresi tra 4,9 e 9,4.

Con una scelta di 16 *thread* CPU per i test su un insieme di genomi, i fattori di *speedup* oscillano tra 5,9 e 9,6. Sull'intera *pipeline* di Flye questo ha un impatto sul tempo di esecuzione che varia da una riduzione del 3% ad una del 39%.

Mediamente, sui dataset utilizzati, il tempo che occupa la fase di correzione delle bolle nell'intera *pipeline* scende dal 19,2% al 3,6%. Il passo ora più ovvio è quindi provare ad applicare lo stesso tipo di *porting* su altre fasi di Flye.

La tesi è strutturata nel seguente modo: nel primo capitolo presento il problema dell'assemblaggio genomico con *read* lunghe; nel secondo descrivo lo stato dell'arte e nello specifico l'assemblatore Flye; nel terzo capitolo spiego la fase di *polishing* di Flye, su cui si concentra il lavoro svolto; nel quarto capitolo illustro l'approccio che ho adottato per fare il *porting* del *polishing* su GPU e come lo ho implementato; nel quinto capitolo discuto i risultati che ho ottenuto; nel sesto espongo quali sono i possibili sviluppi futuri per ridurre ulteriormente i tempi di esecuzione di Flye.

Indice

Abstract **i**

Indice **iii**

1 Introduzione **1**
1.1 Il problema dell'assemblaggio genomico 1
1.2 Regioni ripetute e *read* lunghe 2
1.3 Accelerazione GPU 3
1.4 Obiettivi e struttura della tesi 3

2 Stato dell'arte **4**
2.1 I grafi di de Bruijn 4
2.2 L'approccio A-Bruijn 5
2.3 Flye 6

3 Il *polishing* in Flye **9**
3.1 Dalle *read* alle bolle 9
3.2 Il modello probabilistico 9
3.3 Le matrici *forward* e *reverse* 10
3.4 Utilizzo delle matrici *forward* e *reverse* per calcolare lo score di una modifica 11
3.5 La ricerca della sequenza consenso 11
3.6 Parallelismo 12

4 Implementazione **13**
4.1 Panoramica 13
4.2 Organizzazione del lavoro sulla CPU 14
4.2.1 Lo stato di una bolla 14
4.2.2 Il ciclo del *worker* 15
4.2.3 La divisione in lotti 16
4.2.4 L'elaborazione di un lotto 17
4.3 Il passo 18

4.3.1 *Kernel* di calcolo delle matrici di allineamento 18
4.3.2 I *kernel* di valutazione delle modifiche 19
4.3.3 *Kernel* di riduzione 19

5 Testing e risultati **21**
5.1 Risultati in base al numero di *thread worker* utilizzati 22
5.2 Guadagno su altri genomi 23
5.3 La *pipeline* intera 23

6 Conclusioni e sviluppi futuri **25**
6.1 Rifinitura del software 25
6.2 Una visione sull'intera *pipeline* 26

Bibliografia **27**

Capitolo 1

Introduzione

1.1 Il problema dell'assemblaggio genomico

Il genoma di un organismo è la sequenza completa del suo DNA, composta da basi azotate (adenina, citosina, guanina e timina).

Le tecnologie di sequenziamento non sono in grado di leggerlo dall'inizio alla fine in un'unica operazione. Queste tecnologie frammentano numerose copie dello stesso genoma e per ogni base dei frammenti misurano un segnale fisico, luminoso o elettrico, in base alla tecnologia usata. Il segnale viene poi convertito in un carattere sull'alfabeto {A, C, G, T}. Le stringhe composte da questi caratteri vengono chiamate *read*, lunghe da un centinaio di basi a qualche centinaio di migliaia, raramente oltre il milione [1].

L'assemblaggio genomico è il processo di ricostruzione della sequenza originale a partire da questo insieme disordinato di *read* [2]. Il problema di questo processo è che dai soli dati delle *read* non sappiamo dove esse possano collocarsi nel genoma.

Per risolvere questo problema si utilizza un *assemblatore*, un algoritmo o un software progettato per trovare l'ordine originario delle sequenze. Affinché questo processo sia possibile, è necessaria un'elevata ridondanza dei dati. Il valore di *coverage* (o copertura) specifica quanto il genoma sia sovracampionato. Senza letture ridondanti e sfalsate, sarebbe impossibile concatenare i frammenti in modo continuo [1].

Questo lavoro di tesi riguarda l'assemblatore Flye, un assemblatore *de novo*, ovvero che, a differenza di assemblaggi guidati da un genoma di riferimento (*reference-guided assembly*), ricostruisce il genoma solo dalle sovrapposizioni tra le *read* [1].

Il motivo per cui si usano assemblatori *de novo* è la possibilità di assemblare il primo genoma di una specie o di assemblare genomi con mutazioni.

1.2 Regioni ripetute e *read* lunghe

Uno dei principali problemi dell'assemblaggio *de novo* è costituito dalla presenza delle regioni ripetute, ovvero sequenze di DNA che compaiono più volte all'interno del genoma in copie identiche o altamente simili. Quando una *read* ricade interamente all'interno di una di queste regioni, non fornisce nessun elemento per stabilire a che tratto del genoma appartenga. Di conseguenza, l'assemblatore non è in grado di determinare quali tratti di genoma precedano e seguano la ripetizione, portando a errori nella ricostruzione [2].

Le ripetizioni si dividono in due categorie, che pongono problemi diversi [1]. Le ripetizioni *disperse* si trovano distribuite in posizioni non contigue tra loro, e per ogni ripetizione, se non si ha una *read* che la include completamente, non sarà possibile sapere qual è il prefisso e qual è il suffisso di questa specifica copia tra i suffissi ed i prefissi di tutte le copie della ripetizione. Le ripetizioni *in tandem*, invece, si susseguono consecutivamente l'una dopo l'altra. In quest'ultimo caso, se nessuna *read* è in grado di attraversare l'intera sequenza ripetuta è difficile stabilire il numero esatto di copie consecutive presenti [2].

La capacità di risolvere queste ripetizioni dipende dalla lunghezza delle *read*. Le tecnologie di sequenziamento di seconda generazione, producono *read* corte, dell'ordine del centinaio di basi [1], con un tasso di errore inferiore all'1% [3]. Con *read* così corte molte ripetizioni non vengono attraversate interamente da nessuna *read*, e gli assemblaggi risultano frammentati [1].

Per superare questo limite sono state introdotte le tecnologie di *Single-Molecule Sequencing* (SMS), sviluppate da aziende come Pacific Biosciences e Oxford Nanopore [2]. Queste tecnologie generano *read* lunghe da decine a centinaia di migliaia di basi [1]. Una *read* più lunga di una ripetizione copre la ripetizione per intero ed avrà prefisso e suffisso nelle sequenze uniche ai due lati della ripetizione, permettendo così di stabilirne la sua posizione. Le ripetizioni più lunghe di qualsiasi *read*, però, restano difficili da risolvere [2].

Il principale svantaggio di queste tecnologie è che il tasso di errore delle *read* supera il 10%, un livello a cui gli algoritmi sviluppati per le *read* corte smettono di funzionare e che ha richiesto nuovi algoritmi di assemblaggio [3].

Un assemblaggio costruito a partire dalle *read* ne eredita gli errori, pur avendo una struttura corretta su larga scala [3].

Per correggere gli errori portati dalle tecnologie di sequenziamento si sfrutta la ridondanza dei dati vista nella Sezione 1.1. Poiché il genoma è sovracampionato, ogni sua posizione compare in molte *read* diverse, e gli errori di sequenziamento, in larga parte casuali, difficilmente si ripetono nella stessa posizione su più *read* [1].

Definiamo quindi come *polishing* il processo di correzione degli errori delle *read* rimasti nella bozza finale dell'assemblaggio.

Questo processo ha un costo computazionale significativo, nelle misure degli autori di Flye il *polishing* occupa tra il 37% e il 55% del tempo complessivo dell'assemblaggio [2, Nota supplementare 1].

1.3 Accelerazione GPU

Flye tenta di risolvere il problema dell'elevato costo computazionale della fase di *polishing* lavorando su più *thread* CPU. Il collo di bottiglia perciò risiede nel limitato numero di *core* CPU. Per l'assemblaggio di un genoma umano, Flye ha richiesto complessivamente circa 5.000 ore CPU [2, Nota supplementare 1].

Per ridurre questi tempi di calcolo, si può ricorrere all'accelerazione tramite *Graphics Processing Unit*, o GPU. A differenza delle CPU, le GPU sono progettate per il calcolo massivamente parallelo, con migliaia di *core* in grado di eseguire simultaneamente la stessa istruzione su lunghe sequenze di dati [4].

1.4 Obiettivi e struttura della tesi

L'obiettivo della tesi consiste nella progettazione e nell'implementazione di un *porting* su architettura GPU del passaggio di correzione delle bolle della fase di *polishing* dell'assemblatore, più nel dettaglio il lavoro di tesi consiste nello spostare l'elaborazione di ogni bolla, operazione originariamente eseguita su CPU, su GPU. Sfruttando il calcolo massivamente parallelo, si intende ridurre i tempi di esecuzione del software, mantenendo invariata l'accuratezza dell'output.

Capitolo 2

Stato dell'arte

Flye è uno dei software più recenti basato sui grafi di de Bruijn, lo strumento più diffuso per assemblare le *read* corte [1].

2.1 I grafi di de Bruijn

Nell'approccio basato sui grafi di de Bruijn ogni *read* viene frammentata in tutte le sue sottostringhe di lunghezza fissa k, denominate k-meri, e si costruisce un grafo in cui ogni k-mero è un arco che collega due vertici: il prefisso e il suffisso del k-mero stesso, entrambi di lunghezza $k-1$. Due k-meri consecutivi si sovrappongono per $k-1$ basi, quindi il suffisso del primo coincide con il prefisso del secondo e saranno rappresentati dallo stesso vertice [3].

Il secondo step dell'assemblaggio è la ricerca di un cammino euleriano che percorra quindi tutti gli archi una sola volta. Questo cammino, se trovato, rappresenterebbe l'intero genoma. Ma problemi nella costruzione del grafo portano spesso a situazioni in cui un cammino perfetto non è trovabile. Quello che si fa allora è cercare tratti di cammino che rappresentano segmenti contigui del genoma, detti *contig* [2].

Il grafo di de Bruijn offre una rappresentazione compatta delle regioni ripetute. Due copie identiche di una ripetizione producono lo stesso insieme di k-meri e vengono rappresentate perciò sui medesimi archi [2].

Il vincolo di questo approccio riguarda la scelta di k. Poiché una ripetizione più corta di k non genera ramificazioni, perché ogni k-mero che la contiene include anche un tratto della regione unica che la precede o la segue, il problema delle ripetizioni sarebbe più facilmente risolvibile per k più grandi. Più k è grande, quindi, meno il grafo è ramificato. D'altra parte, più k è grande, più è probabile che un k-mero contenga almeno un errore di lettura, e affinché il grafo risulti connesso è indispensabile che la maggior parte dei k-meri del genoma compaia intatta in molte *read* [2]. Per cui nei grafi di de Bruijn la scelta di k dipende dal tasso di errore delle letture.

2.2 L'approccio A-Bruijn

Le *read* prodotte dal sequenziamento a singola molecola sembrano adatte a sfruttare al meglio i grafi di de Bruijn, lavorando tra decine e centinaia di migliaia di basi, consentirebbero in principio di usare k-meri molto lunghi e di ottenere grafi quasi privi di ramificazioni. Queste tecnologie hanno però tassi di errore superiori al 10%, che come abbiamo visto limitano profondamente la funzionalità dei grafi de Bruijn [3].

Se p è la probabilità che una base sia errata, la probabilità che un k-mero si conservi intatto è $(1-p)^k$. Con $p = 0{,}1$, un k-mero di 15 basi è intatto in circa il 20% dei casi, uno di 25 basi in circa il 7%. Il grafo di de Bruijn risulta quindi inutilizzabile [3].

Per aggirare questo limite è stato introdotto il grafo A-Bruijn, una versione del grafo di de Bruijn che non utilizza tutti i k-meri disponibili, ma solo un sottoinsieme di sequenze chiamate *solid string*, ovvero i k-meri che compaiono con elevata frequenza nelle *read* come mostrato dalla Figura 1. Partendo dal presupposto che gli errori di sequenziamento siano distribuiti casualmente e raramente si ripetano nella stessa posizione su più *read*, due *read* che si sovrappongono condividono molte *solid string*, e l'algoritmo le usa per individuare le sovrapposizioni tra *read* rumorose [3].

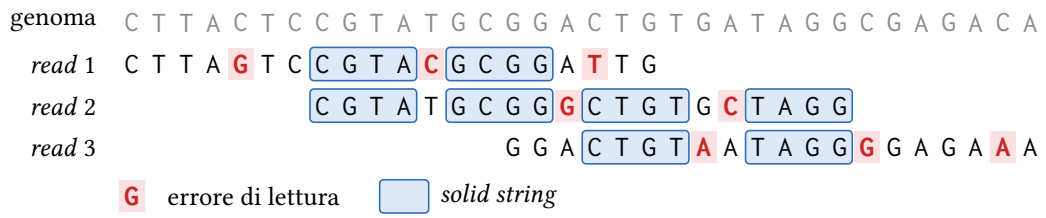


Figura 1: *Solid string* in un esempio con $k = 4$, in cui sono *solid* i 4-meri presenti in almeno due *read*, in tre *read* estratte dallo stesso tratto di genoma, con gli errori di lettura in rosso.

Il grafo ottenuto unendo le *read* sarà topologicamente corretto ma conterrà ancora gli errori delle letture, e A-Bruijn li corregge con una fase di *polishing* [3], punto centrale di questa tesi, che verrà descritta nel Capitolo 3 e da cui deriva quella di Flye [5].

L'approccio di A-Bruijn presenta un limite, l'algoritmo si ferma prima di costruire il genoma completo dal grafo delle ripetizioni, la struttura del quale ha spesso una topologia troppo complessa per poterne estrapolare tutti i *contig*. È questo il motivo per cui Flye utilizza un altro approccio, rinuncia alla costruzione esatta del grafo per generarne invece uno approssimato [2].

2.3 Flye

Gli assemblatori tradizionali tentano di generare subito sequenze continue e accurate, i *contig*, percorrendo il grafo costruito dalle *read*. Flye si comporta diversamente, la prima fase del suo algoritmo produce i *disjointig* (Figura 2), catene di *read* unite in base alla sovrapposizione arbitraria tra suffissi e prefissi combacianti, senza verificare che ogni unione sia quella corretta. Un *disjointig* può quindi unire tratti di genoma non contigui, ma generarlo è molto più rapido che costruire un *contig* accurato [2].

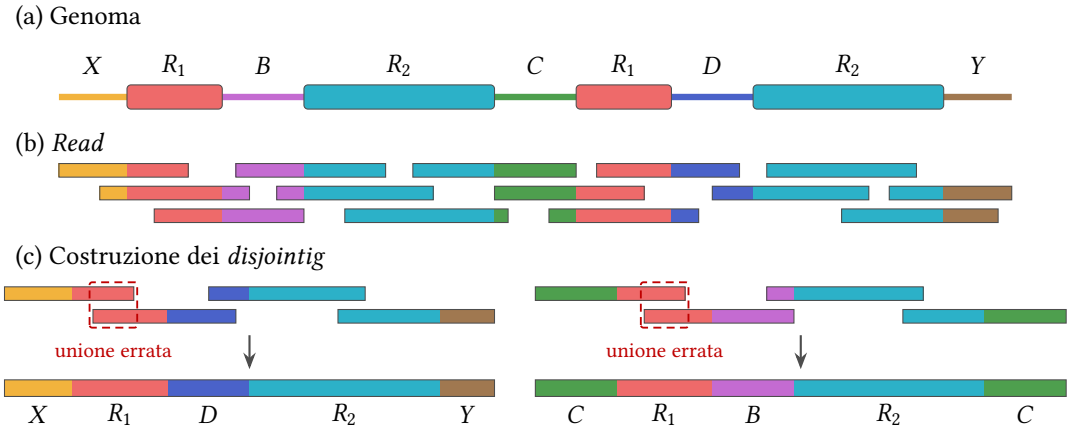


Figura 2: Costruzione dei *disjointig*. (a) Un genoma con due ripetizioni, R_1 e R_2, presenti in due copie ciascuna e separate dalle regioni uniche X, B, C, D e Y. (b) *Read* estratte dal genoma. (c) Costruzione dei *disjointig*.

La sequenza di ogni *disjointig* contiene gli stessi errori delle *read* da cui è formata, e viene poi corretta con un voto di maggioranza, Flye allinea sul *disjointig* tutte le *read* e in ogni posizione sceglie la base indicata dalla maggior parte di esse [2, Nota supplementare 8].

Una volta generati, i *disjointig* vengono concatenati in un'unica stringa, separati da appositi delimitatori, andando a costituire uno pseudo-genoma. Flye allinea questa stringa con sé stessa per individuare i tratti che si ripetono e, a partire dai punti estremi di questi auto-allineamenti, costruisce un *grafo delle ripetizioni approssimato* (*approximate repeat graph*). Il grafo ottenuto rappresenta correttamente le ripetizioni del genoma [2] come mostrato dalla Figura 3 riprendendo il genoma presentato dalla Figura 2.

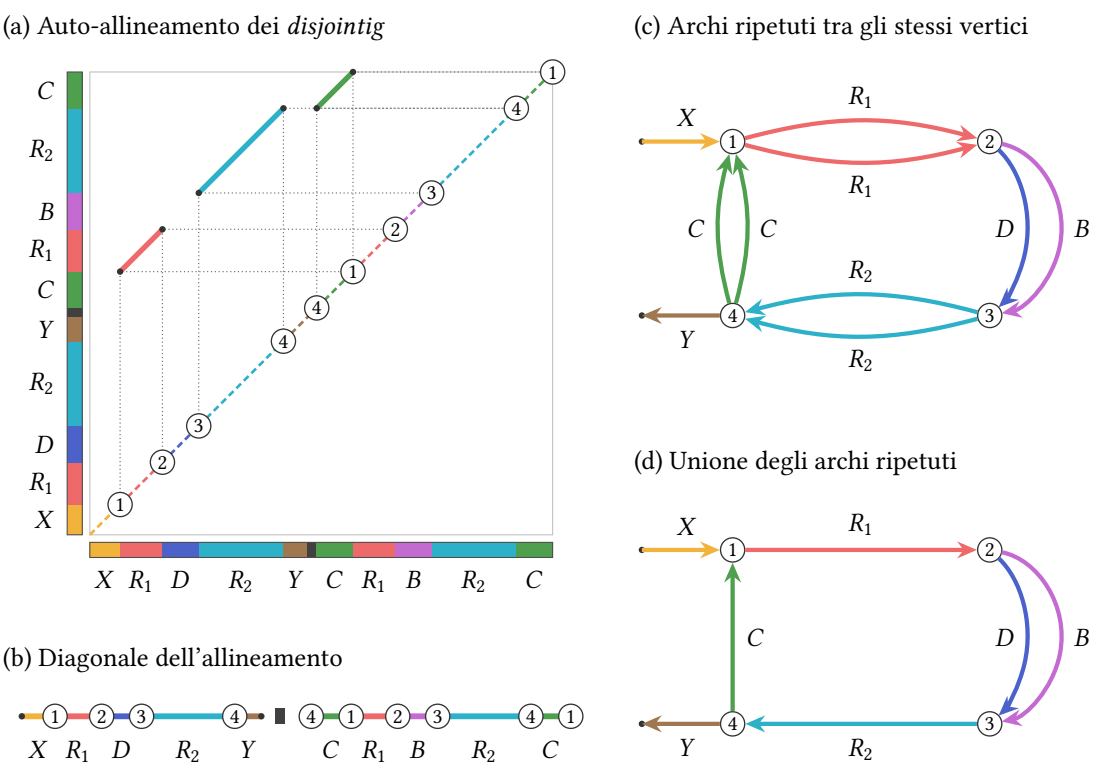


Figura 3: Costruzione del grafo delle ripetizioni dai *disjointig* della Figura 2. (a) I due *disjointig* con un delimitatore (in nero) sono allineati con sé stessi. (b) La diagonale dell'allineamento: le proiezioni rappresentano archi del grafo delle ripetizioni da costruire. (c) Proiezioni dello stesso tipo rappresentano archi ripetuti tra gli stessi vertici. (d) Gli archi ripetuti vengono uniti.

Ottenuto il grafo, l'assemblatore procede a districarlo allineando su di esso l'intero set di *read* originali. Questo passaggio permette di risolvere le *bridged repeat*, ovvero le regioni ripetute più corte di almeno una delle *read* a disposizione. Una *read* di questo tipo attraversa l'intera ripetizione ancorandosi alle regioni uniche ai due lati, e indica così come i vertici del grafo debbano essere collegati [2] (Figura 4).

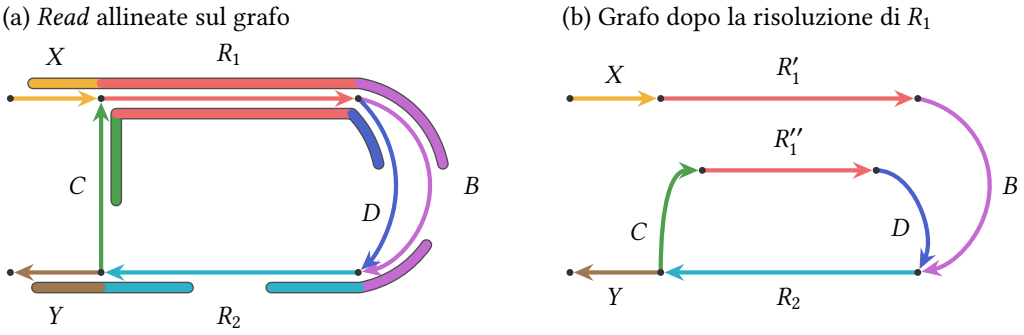


Figura 4: Risoluzione di una *bridged repeat* nel grafo della Figura 3. (a) Le *read* allineate sul grafo. (b) *Bridged repeat* risolte.

Al termine di questo passaggio l'algoritmo estrae i *contig*, che corrispondono ai cammini non ambigui all'interno del grafo delle ripetizioni, come nei grafi de Bruijn, e vi applica il *polishing* per correggere gli errori ereditati dalle *read* [2].

Capitolo 3

Il *polishing* in Flye

I *contig* estratti dal grafo delle ripetizioni hanno una struttura corretta, ma la loro sequenza deriva dal voto di maggioranza con cui Flye costruisce i *disjointig*, che porta il tasso di errore dal 13% circa delle *read* grezze a valori compresi tra l'1% e il 5%. Il *polishing*, ultima fase della *pipeline*, lo riduce a circa lo 0,1% quando la copertura supera 30×, ovvero che in media ogni base del genoma è contenuta in 30 *read* diverse [2, Nota supplementare 8].

3.1 Dalle *read* alle bolle

Flye comincia allineando tutte le *read* alla bozza del genoma con minimap2 [6]. Si scelgono poi dei punti di taglio, in zone in cui le *read* concordano con la bozza, per suddividerla in sezioni più piccole. Ogni tratto compreso tra due tagli consecutivi forma una bolla, che contiene la sequenza candidato, ovvero la sequenza tra i due tagli nella bozza, e i tratti delle *read* allineati allo stesso intervallo, chiamati rami. In più, Flye scarta i rami di lunghezza molto diversa dalla mediana della bolla e ne tiene al massimo 50 [5]. Lo schema è quello di A-Bruijn, in cui i tratti tra due tagli consecutivi sono lunghi in media circa 30 basi [3].

L'output di questo processo, le bolle, viene scritto su un file, da cui il *polisher* le legge e le corregge indipendentemente l'una dall'altra. Le bolle corrette vengono poi ordinate per posizione e concatenate nei nuovi *contig*.

3.2 Il modello probabilistico

Definito l'allineamento tra due stringhe come la sovrapposizione tra esse carattere per carattere, atto a confrontare le differenze tra le due, e un gap come un carattere "vuoto" usato per rappresentare la mancanza di corrispondenza tra una zona di una delle due stringhe e l'altra.

Ogni coppia di caratteri $x \in c, y \in r$ appartenente ad un allineamento tra il candidato c e un ramo r, in cui x e y sono una base o un gap, ha una probabilità $p(x, y)$, che dipende dal tipo di evento (corrispondenza, sostituzione, cancellazione o inserimento) e dalla tecnologia di sequenziamento [3]. Flye la converte in un punteggio intero [5]:

$$score(x, y) = \text{round}\left(2^{17} \ln p(x, y)\right)$$

Lo score di un ramo r è quindi il punteggio del miglior allineamento A tra il candidato c e il ramo stesso, cioè la somma dei punteggi delle coppie di caratteri di A, che approssima il logaritmo del prodotto delle probabilità:

$$Score(c, r) = \max_A \sum_{(x,y)\in A} score(x, y) \approx 2^{17} \ln \max_A \prod_{(x,y)\in A} p(x, y)$$

Lo score di un candidato c è quindi la somma degli score di tutti i rami ad esso associati, e la sequenza consenso è una delle sequenze di caratteri che massimizzano questa somma:

$$Score(c) = \sum_{j=1}^{B} Score(c, r^{(j)})$$

Per ogni bolla, si cerca quindi la sequenza *consenso* che massimizza lo score dell'allineamento tra se stessa e i rami.

3.3 Le matrici *forward* e *reverse*

Lo score di un ramo si calcola in Flye con l'algoritmo di Needleman–Wunsch [7]. Flye lo calcola in entrambe le direzioni perché, le due matrici ottenute permettono di valutare una modifica senza ricalcolare l'intero allineamento; A-Bruijn usa lo stesso calcolo *forward–backward* per trovare la modifica migliore con un solo passaggio su ogni ramo [3].

Indicando con $c_1 \dots c_n$ il candidato e con $r_1 \dots r_m$ il ramo, la cella $F(i, k)$ della matrice di allineamento F, di dimensioni $(n + 1) \times (m + 1)$, contiene il punteggio del miglior allineamento tra le prime i basi del candidato e le prime k basi del ramo.

Si denominano F la matrice *forward* così ottenuta e R la matrice *reverse*, calcolata con lo stesso algoritmo sulle due sequenze invertite: $R(i, k)$ è il punteggio del miglior allineamento tra le ultime i basi del candidato e le ultime k basi del ramo.

3.4 Utilizzo delle matrici *forward* e *reverse* per calcolare lo score di una modifica

Indico con c' il candidato c modificato, con p la posizione della modifica e con x la nuova base, nel caso di sostituzioni e inserimenti.

In un allineamento tra c' e un ramo r di lunghezza m, le prime k basi del ramo, per un certo k tra 0 e m, sono allineate alla parte di c' che precede la modifica, e le restanti $m-k$ basi del ramo sono allineate alla parte di c' successiva alla modifica.

Fissato k, le due parti si allineano indipendentemente, e lo score del miglior allineamento di ciascuna è già stato calcolato rispettivamente nella matrice F e nella matrice R.

Indichiamo con $V(k)$ lo score del miglior allineamento tra c' e r in cui le prime k basi del ramo sono allineate alla parte di c' precedente alla modifica:

- cancellazione di c_p:

$$V(k) = F(p-1,\ k) + R(n-p,\ m-k);$$

- sostituzione di c_p con x:

$$V(k) = \max\{F(p-1,\ k-1) + score(x, r_k),\ F(p-1,\ k) + score(x, -)\} + R(n-p,\ m-k);$$

- inserimento di x prima di c_p:

$$V(k) = \max\{F(p-1,\ k-1) + score(x, r_k),\ F(p-1,\ k) + score(x, -)\} + R(n-p+1,\ m-k).$$

Voglio quindi trovare k che mi massimizzi $V(k)$ per trovare lo score del miglior allineamento tra r e c'

$$Score(c', r) = \max_{0 \le k \le m} V(k)$$

Ogni $V(k)$ richiede un numero costante di operazioni, quindi valutare una modifica costa $O(m)$ operazioni per ramo, invece delle $O(nm)$ di un nuovo allineamento.

3.5 La ricerca della sequenza consenso

Il fulcro del *polisher*, nel codice del programma, è il metodo `makeStep`, che esegue un passo della ricerca: calcola le matrici F e R di ogni ramo, valuta le modifiche di una sola base e restituisce il candidato con la modifica migliore, oppure lo lascia invariato se nessuna modifica ne migliora il punteggio [5].

Le $8n + 4$ modifiche possibili vengono valutate in tre gruppi, in ordine fisso: prima le n cancellazioni, poi i $4(n+1)$ inserimenti, uno per ogni base, infine le $3n$ sostituzioni, una

per ogni base diversa dall'originale. Se un gruppo contiene una modifica che migliora il punteggio, viene restituita la migliore del gruppo, senza esaminare i gruppi successivi; a parità di punteggio vince la prima incontrata.

Il *polishing* di una bolla chiama `makeStep` iterativamente, ogni volta sul candidato c restituito dal passo precedente. Il processo si interrompe quando nessuna modifica migliora lo score di c oppure quando il numero di chiamate a `makeStep` arriva a un numero pari a dieci volte la lunghezza iniziale del candidato.

Se la bolla ha più di dieci rami, il ciclo viene eseguito due volte: la prima, detta *pre-polish*, usa solo i cinque rami di lunghezza più vicina alla mediana, la seconda usa tutti i rami e parte dal candidato ottenuto [5].

3.6 Parallelismo

Il calcolo offre parallelismo a più livelli. Le bolle sono indipendenti, e la versione CPU sfrutta questo livello con i *thread*. All'interno di un passo, le matrici dei diversi rami si calcolano indipendentemente, e una volta calcolate le matrici anche le valutazioni delle modifiche sono indipendenti tra loro, ma la versione CPU le esegue in sequenza. Il limite è la sequenzialità delle chiamate a `makeStep`, ognuna delle quali parte dal candidato restituito dalla precedente.

Capitolo 4

Implementazione

4.1 Panoramica

Il lavoro svolto è stato il *porting* del processo di correzione delle bolle della fase di *polishing* da codice C++ a CUDA, il modello di programmazione sviluppato da NVIDIA per le proprie GPU [4]. In CUDA la CPU è detta *host* e la GPU *device*, ciascuna con la propria memoria. Le funzioni che l'*host* fa eseguire al *device* si chiamano *kernel*. Lanciare un *kernel* significa avviare sul *device* i *thread* che ne eseguono il codice in parallelo, raggruppati in blocchi. Tutti i *thread* di un blocco sono eseguiti dallo stesso *Streaming Multiprocessor*, l'unità di computazione principale della GPU.

Definiamo come *thread worker* o *worker* i *thread* CPU che si suddividono il lavoro per distinguerli dai *thread* della GPU.

Ogni *worker* tiene in elaborazione un *pool* di al massimo 32 bolle, deciso in fase di testing, e lavora a *round*: in ogni *round* ciascuna bolla del *pool* esegue un passo, paragonabile a `makeStep` in Flye. Per limitare la memoria occupata sulla GPU, ad ogni step il *pool* viene diviso in *lotti*, gruppi di bolle di cui la GPU esegue il passo contemporaneamente. Vengono elaborati sequenzialmente uno dopo l'altro.

Le bolle che concludono il *polishing* vengono liberate dal *pool*, e al *round* successivo il loro posto viene preso da nuove bolle.

Il criterio che ho seguito nel *porting* a GPU è l'equivalenza di output con Flye: per ogni bolla, il *backend* GPU deve produrre la stessa sequenza consenso della versione CPU. La Sezione 4.2 descrive l'organizzazione del lavoro sulla CPU, la Sezione 4.3 i *kernel* che eseguono il passo sulla GPU.

4.2 Organizzazione del lavoro sulla CPU

4.2.1 Lo stato di una bolla

Con l'organizzazione a *round* il *worker* esegue un solo passo per bolla e, al *round* successivo, deve riprendere ciascuna bolla dal punto in cui l'aveva lasciata. Lo stato di ogni bolla viene quindi conservato in un oggetto che sopravvive da un *round* all'altro (Tabella 1).

Tabella 1: Oggetto che descrive lo stato di una bolla.

Campo	Contenuto
`bubble`	• il *contig* e la posizione a cui appartiene • il candidato iniziale • i rami
`candidate`	il candidato corrente
`reducedSet`	una copia dei rami usati nel *pre-polish*
`branches`	i rami del passo corrente: punta a `reducedSet` o ai rami della bolla
`phase`	la fase: 0 *pre-polish*, 1 tutti i rami, 2 conclusa
`iterNum`	i passi eseguiti nella fase corrente
`phaseStartSize`	la lunghezza del candidato all'inizio della fase
`cells`	le celle delle matrici del passo corrente

L'oggetto viene creato quando la bolla entra nel *pool*: se la bolla ha più di dieci rami, i cinque usati nel *pre-polish* vengono copiati in `reducedSet` e la bolla parte dalla fase 0, altrimenti parte dalla fase 1.

Dopo ogni passo, la GPU produce gli score delle modifiche e il *worker* applica le condizioni di arresto, pari a quelle di `makeStep`, usando `iterNum` e `phaseStartSize` per il limite sul numero di passi. Se nessuna condizione termina il processo, la bolla fa un altro passo di *polishing* aggiornando `candidate` con quello prodotto dal passo precedente, altrimenti si passa alla fase successiva.

Al termine di una fase l'elaborazione avanza come nel diagramma della Figura 5. Alla fine del *pre-polish* `iterNum` e `phaseStartSize` vengono reimpostati. `branches` punta ai rami della bolla e si passa alla fase 1.

Finita la fase 1 il candidato diventa il consenso della bolla, si passa alla fase 2 e viene tolta dal *pool* al termine del *round*.

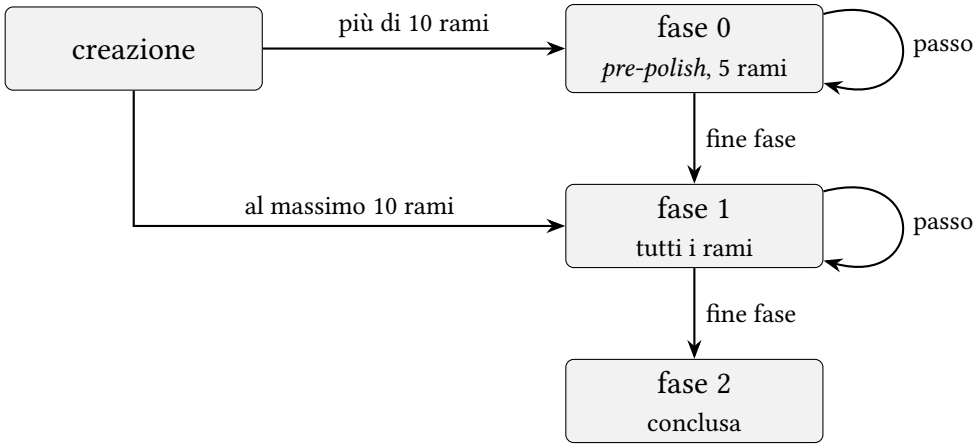


Figura 5: Diagramma delle fasi di una bolla.

Il flusso di controllo di ogni bolla è quindi identico a quello di Flye: stessa preparazione, stesse condizioni di arresto e stesse transizioni di fase.

4.2.2 Il ciclo del *worker*

I *worker* usano le stesse risorse condivise e lo stesso *mutex* dei *worker* di Flye [5], ma gestiscono *pool* di bolle invece di bolle singole.

Ogni *worker* possiede inoltre una lista di bolle di cui ha concluso il *polishing* in attesa dell'acquisizione del *mutex* per essere scritte su file di output, e un insieme di *buffer* sul *device* per memorizzare i dati dei suoi lotti.

Algoritmo 1 Ciclo di un *worker*.

```
1  ripeti:
2      // 1. sezione critica, con il mutex
3      acquisisci il mutex
4      scrivi nel file di uscita le bolle concluse
5      finché l'input non è esaurito e il pool contiene meno di 32 bolle:
6          se la cache è vuota: riempila leggendo cento bolle dal file
7          preleva una bolla dalla cache
8          crea l'oggetto che ne descrive lo stato e inseriscilo nel pool
9      rilascia il mutex
10
11     se il pool è vuoto: esci dal ciclo
12
13     // 2. round, senza mutex
14     esegui un passo per ogni bolla del pool
15
16     // 3. bolle concluse, senza mutex
17     per ogni bolla del pool in fase 2:
18         spostala tra le bolle concluse e toglila dal pool
```

Anche qui le bolle vengono scritte nell'ordine in cui concludono, senza effetti sul risultato.

4.2.3 La divisione in lotti

Per ogni bolla un passo richiede le matrici *forward* e *reverse* di ogni ramo. All'interno di un lotto, le matrici di tutte le bolle devono trovarsi contemporaneamente nella memoria del *device*: il primo *kernel* le scrive tutte, e solo dopo i *kernel* di valutazione delle modifiche le leggono. Per contenere la memoria, dividiamo il *round* in lotti le cui matrici occupano al massimo un numero prestabilito di byte, nel nostro caso 128 MiB.

Per ogni bolla b viene quindi calcolato il numero di celle delle matrici richieste dal passo:

$$celle(b) = \sum_{j=1}^{B} (n+1)(m_j+1) \tag{1}$$

dove B è il numero di rami in uso, n la lunghezza del candidato e m_j quella del ramo $r^{(j)}$. Il conteggio si riferisce a una sola direzione: le matrici *forward* e *reverse* hanno lo stesso numero di celle e ogni cella occupa 8 byte, 16 quindi per ogni coppia di celle R e F.

Le bolle vengono poi raggruppate scorrendo il *pool* in ordine, con la strategia *next fit* (Algoritmo 2).

Algoritmo 2 Divisione di un *round* in lotti.

per ogni bolla b del pool: calcola $celle(b)$
finché restano bolle da assegnare:
  apri un lotto con la prossima bolla b
  $totale \leftarrow celle(b) \cdot 16$
  finché esiste una bolla successiva b' e $totale + celle(b') \cdot 16 \leq limite$:
    aggiungi b' al lotto
    $totale \leftarrow totale + celle(b') \cdot 16$
  esegui il passo per le bolle del lotto

Ogni lotto contiene almeno una bolla. Una bolla le cui matrici superano da sole il limite forma quindi un lotto a sé, ed è l'unico caso in cui le matrici di un lotto superano i 128 MiB.

La suddivisione in lotti viene ricalcolata a ogni *round*, perché la memoria richiesta da ogni bolla cambia nel tempo: il candidato si allunga o si accorcia a ogni passo, nuove bolle entrano nel *pool* e, nel passaggio dalla fase 0 alla fase 1, una bolla elabora prima solo 5 rami e successivamente tutti quelli che ha a disposizione.

4.2.4 L'elaborazione di un lotto

Ogni lotto viene elaborato in due momenti: prima la GPU esegue il passo per tutte le sue bolle, poi la CPU sceglie per ciascuna bolla la modifica da applicare e ne aggiorna lo stato.

Sul *device* viene calcolato, per tutte le bolle del lotto, ciò che `makeStep` calcola per una sola, tranne la scelta finale della modifica, che resta alla CPU. Il suo funzionamento interno è descritto nella Sezione 4.3. Nell'effettivo, per ogni bolla restituisce:

- lo score $S = Score(c)$ del candidato corrente;
- la migliore cancellazione, con posizione e score della modifica;
- il migliore inserimento, con posizione, base inserita e score della modifica;
- la migliore sostituzione, con posizione, nuova base e score della modifica.

La CPU sceglie poi la modifica con lo stesso ordine fisso di `makeStep`:

Algoritmo 3 Scelta della modifica da applicare.

se la migliore cancellazione supera S: applica la cancellazione
altrimenti se il migliore inserimento supera S: applica l'inserimento
altrimenti se la migliore sostituzione supera S: applica la sostituzione
altrimenti: nessuna modifica

Questo ordine non sceglie necessariamente la modifica migliore in assoluto, ma è stato mantenuto perché i risultati coincidano con quelli di Flye.

4.3 Il passo

Il passo delle bolle di un lotto viene eseguito sulla GPU come una sequenza di 3 operazioni:

1. la prima calcola le matrici *forward* e *reverse* di tutti i rami del lotto e lo score di ogni ramo;
2. la seconda valuta rispettivamente tutte le cancellazioni, tutti gli inserimenti e tutte le sostituzioni possibili con le formule del Capitolo 3, e scrive un punteggio per ogni modifica;
3. l'ultima, la riduzione segmentata, sceglie per ogni bolla la migliore modifica di ciascun tipo e calcola lo score S.

Queste operazioni vengono fatte su GPU tramite lanci di *kernel* CUDA [4].

4.3.1 *Kernel* di calcolo delle matrici di allineamento

Il primo *kernel* calcola le matrici F e R di ogni ramo con l'algoritmo di Needleman e Wunsch [7] (Capitolo 3).

Per sfruttare le capacità di calcolo parallelo della GPU viene però cambiata la sequenza in cui vengono calcolati i valori delle matrici. Invece di essere calcolata riga per riga, ogni matrice viene calcolata a step di 32 colonne alla volta su un *warp*, ovvero un insieme di 32 *thread* GPU che vengono eseguiti assieme.

Ogni cella (i, k) di una matrice di allineamento dipende dalle celle $(i-1, k)$, $(i, k-1)$ e $(i-1, k-1)$. Le celle di una stessa antidiagonale possono essere quindi calcolate assieme poiché sono indipendenti tra loro.

Ad ogni iterazione quindi invece di calcolare una sola cella, calcolo un'antidiagonale di una sezione di 32 colonne della matrice.

Le matrici vengono poi scritte direttamente nella memoria globale del *device*, dove restano a disposizione dei *kernel* di valutazione. Tutte le matrici del lotto si trovano in un unico *buffer*, diviso in due regioni della stessa dimensione: la prima contiene le matrici F, la seconda le R. Le due matrici di un ramo iniziano allo stesso *offset* nelle due regioni, e quelle dei rami di una stessa bolla sono contigue. Ogni matrice è memorizzata per righe: la cella (i, k) di un ramo si trova in posizione $o + i(m+1) + k$, dove o è l'*offset* del ramo e $m+1$ il numero di colonne.

Il formato per righe è scelto in funzione dei *kernel* di valutazione (Sezione 4.3.2). Ognuno di essi legge una riga intera di F e una di R, con *thread* consecutivi su colonne consecutive. Ogni riga viene inoltre letta da più lanci di *kernel*: una riga di F, per esempio, serve alla cancellazione, ai quattro inserimenti e alle tre sostituzioni nella posizione corrispondente.

Oltre alle matrici, il *kernel* scrive lo score di ogni ramo, $F(n, m)$, che la riduzione segmentata somma per ottenere lo score S (Sezione 4.3.3).

4.3.2 I *kernel* di valutazione delle modifiche

I *kernel* di valutazione delle modifiche calcolano gli score $Score(c')$ di ogni modifica applicata a c, la sequenza candidato. La logica di valutazione rimane la stessa di Flye presentata nella Sezione 3.4, per ogni ramo r, $Score(c', r)$ è il massimo dei valori $V(k)$ per k da 0 a m, dove m è la lunghezza del ramo. $Score(c')$ è la somma degli score di tutti i rami.

Le modifiche sono indipendenti l'una dall'altra: Flye le valuta una dopo l'altra, mentre in questo *porting* vengono valutate in parallelo.

Per ciascuno dei tre tipi di modifica (cancellazione, inserimento e sostituzione) viene lanciato un *kernel*, che valuta tutte le modifiche di quel tipo per tutte le bolle del lotto.

Una bolla il cui candidato è lungo n ha $8n + 4$ modifiche da valutare (Sezione 3.5). In un lotto di L bolle le modifiche sono quindi $\sum_{b=1}^{L}(8n_b + 4)$, dove n_b è la lunghezza del candidato della bolla b. Tra gli score calcolati, il *kernel* di riduzione (Sezione 4.3.3) sceglie poi per ogni bolla la migliore modifica di ciascun tipo.

Ogni modifica viene valutata da un blocco di 256 *thread*. Il blocco elabora uno alla volta i rami della bolla su cui sta lavorando. Per ciascun ramo r, di lunghezza m, il blocco procede così:

1. I *thread* si dividono i valori di k da 0 a m secondo lo schema detto *block-stride loop*: il *thread* con indice t nel blocco, con t da 0 a 255, calcola $V(k)$ per $k = t,\ t+256,\ t+512$ e così via, finché $k \leq m$, e conserva soltanto il massimo dei valori che ha calcolato. Per calcolare $V(k)$ il *thread* legge celle di F e R, la base r_k e i punteggi $score(x, \cdot)$, senza modificare nulla, quindi non ha bisogno dei valori calcolati dagli altri *thread*.
2. I 256 *thread* combinano i loro massimi in un unico valore con una riduzione, cioè un'operazione che combina molti valori in uno solo, in questo caso prendendone il massimo. Il valore ottenuto è $Score(c', r)$.
3. Il *thread* con indice 0 somma $Score(c', r)$ al totale dei rami già elaborati.

Dopo l'ultimo ramo il totale è $Score(c')$.

Il *thread* con indice 0 scrive infine $Score(c')$ in memoria globale nello stesso ordine in cui `makeStep` esamina le modifiche, prima per posizione e, a parità di posizione, per base x nell'ordine A, C, G, T.

4.3.3 *Kernel* di riduzione

L'ultimo *kernel*, per ogni bolla del lotto trova, con operazioni di riduzione sui dati prodotti dai *kernel* precedenti:

- la migliore cancellazione;
- il migliore inserimento;
- la migliore sostituzione;
- lo score S del candidato corrente.

A parità di punteggio tra due modifiche dello stesso tipo, viene considerata migliore quella che compare per prima nella sequenza candidato, simulando l'output di `makeStep`.

Oltre allo score S, ognuna delle tre modifiche migliori viene consegnata alla CPU come un punteggio, un indice e, per inserimenti e sostituzioni, la base. Questi risultati sono l'unico dato che torna all'*host*. La CPU confronta poi le tre modifiche con S e sceglie quale applicare (Sezione 4.2.4).

Capitolo 5

Testing e risultati

Il lavoro aveva l'obiettivo di ridurre il tempo di correzione delle bolle, spostando sulla GPU il calcolo di ogni passo del *polishing* (Capitolo 4).

I primi test sono stati svolti sul dataset SRR7594464, che contiene *read* PacBio di *C. elegans*, con numeri diversi di *thread worker*, per osservare come cambia l'accelerazione offerta dalla GPU al variare dei *worker*. Successivamente, scelti 16 *thread worker*, ho eseguito delle run di test su altri sei dataset per confrontare il tempo di esecuzione su CPU con quello su GPU anche su genomi diversi.

Tutti i test sono stati eseguiti su una workstation con la configurazione riportata nella Tabella 2.

Tabella 2: Configurazione della macchina di test.

Componente	**Configurazione**
CPU	AMD Ryzen Threadripper PRO 9975WX
Memoria	250 GiB
GPU	NVIDIA RTX PRO 6000 Blackwell Workstation Edition
Archiviazione	SSD NVMe
Sistema operativo	Rocky Linux 9.8
Software	driver NVIDIA 595.91.07, CUDA Toolkit 13.1, GCC 15.2.0

Come già visto nel Capitolo 4, la versione di Flye su GPU applica al *polishing* di ogni bolla la stessa logica deterministica di Flye. Tutte le esecuzioni in fase di testing del *polishing* su GPU hanno prodotto lo stesso risultato di Flye, e la qualità dell'assemblaggio è rimasta invariata.

5.1 Risultati in base al numero di *thread worker* utilizzati

Quello che si può notare dalle esecuzioni di test con quantità di *thread worker* diverse, è che il tempo di esecuzione del *polishing* su GPU, rispetto a quello su CPU, è decisamente meno dipendente dal numero di *worker* su cui viene fatto eseguire l'algoritmo.

La Figura 6 confronta Flye e la versione GPU nella correzione di tutte le bolle di *C. elegans*, con 8, 12, 16 e 32 *thread worker*. La versione GPU usa *pool* di 32 bolle e lotti fino a 128 MiB.

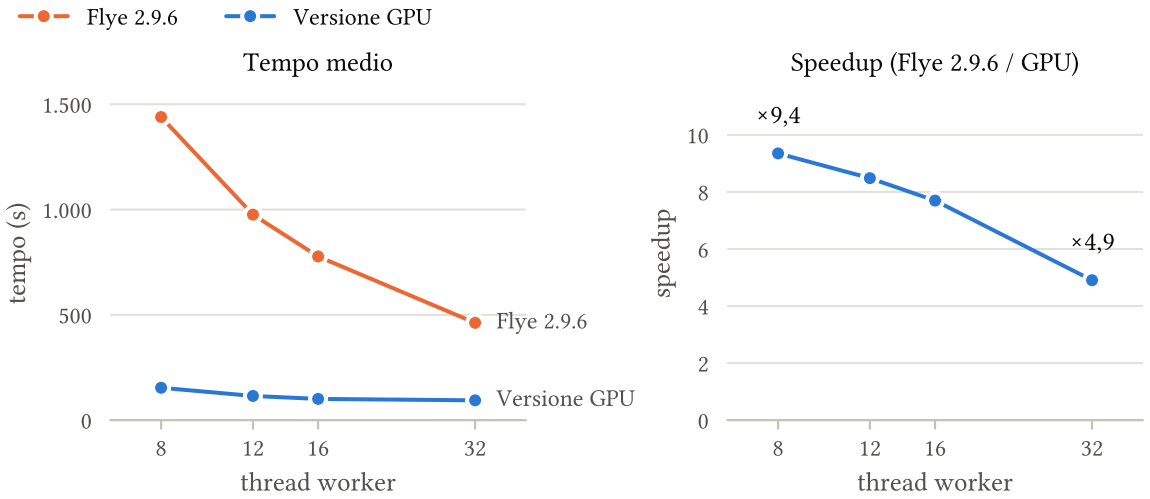


Figura 6: Correzione delle bolle di *C. elegans* con Flye 2.9.6 e con la versione GPU, al variare del numero di *thread worker*.

5.2 Guadagno su altri genomi

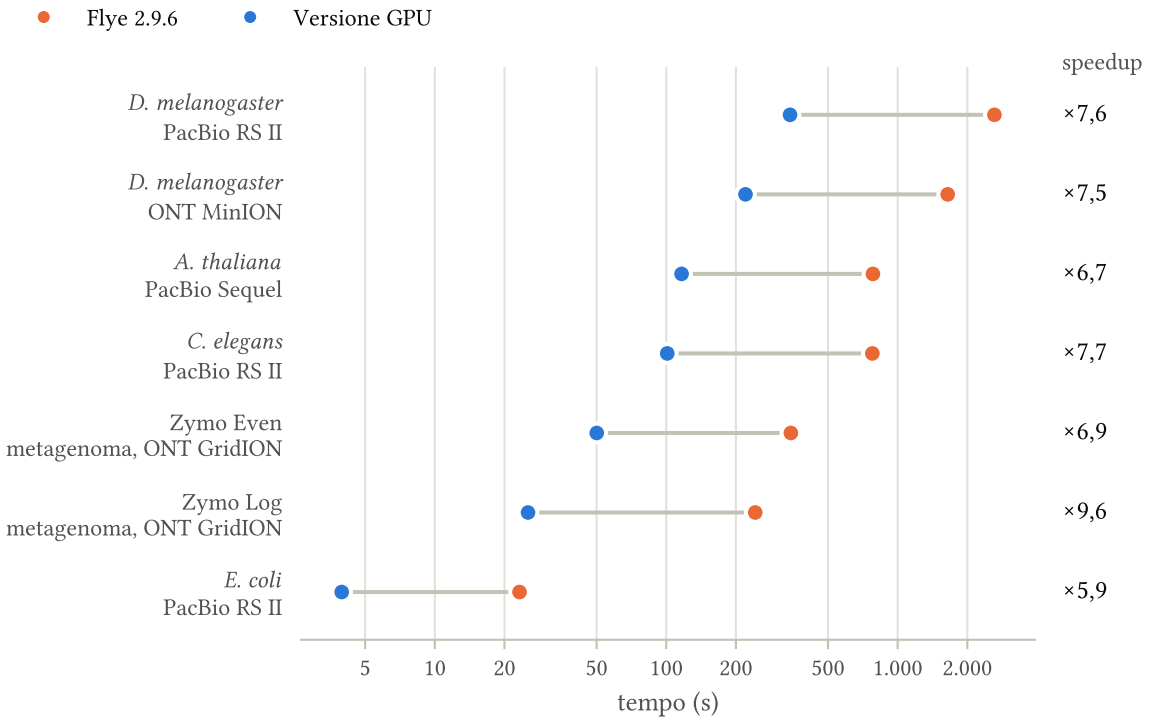


Figura 7: Tempo di correzione di tutte le bolle di diversi genomi con 16 *thread worker*, con Flye 2.9.6 e con la versione GPU.

Come si può vedere dalla Figura 7, la correzione di tutte le bolle, con 16 *thread worker*, sui dataset testati, è da 5,9 a 9,6 volte più veloce rispetto alla versione CPU.

5.3 La *pipeline* intera

Sul tempo complessivo di Flye il guadagno è minore, perché la correzione delle bolle è solo una delle sue fasi: a seconda del dataset, l'intera esecuzione si riduce tra il 3% e il 39% (Figura 8).

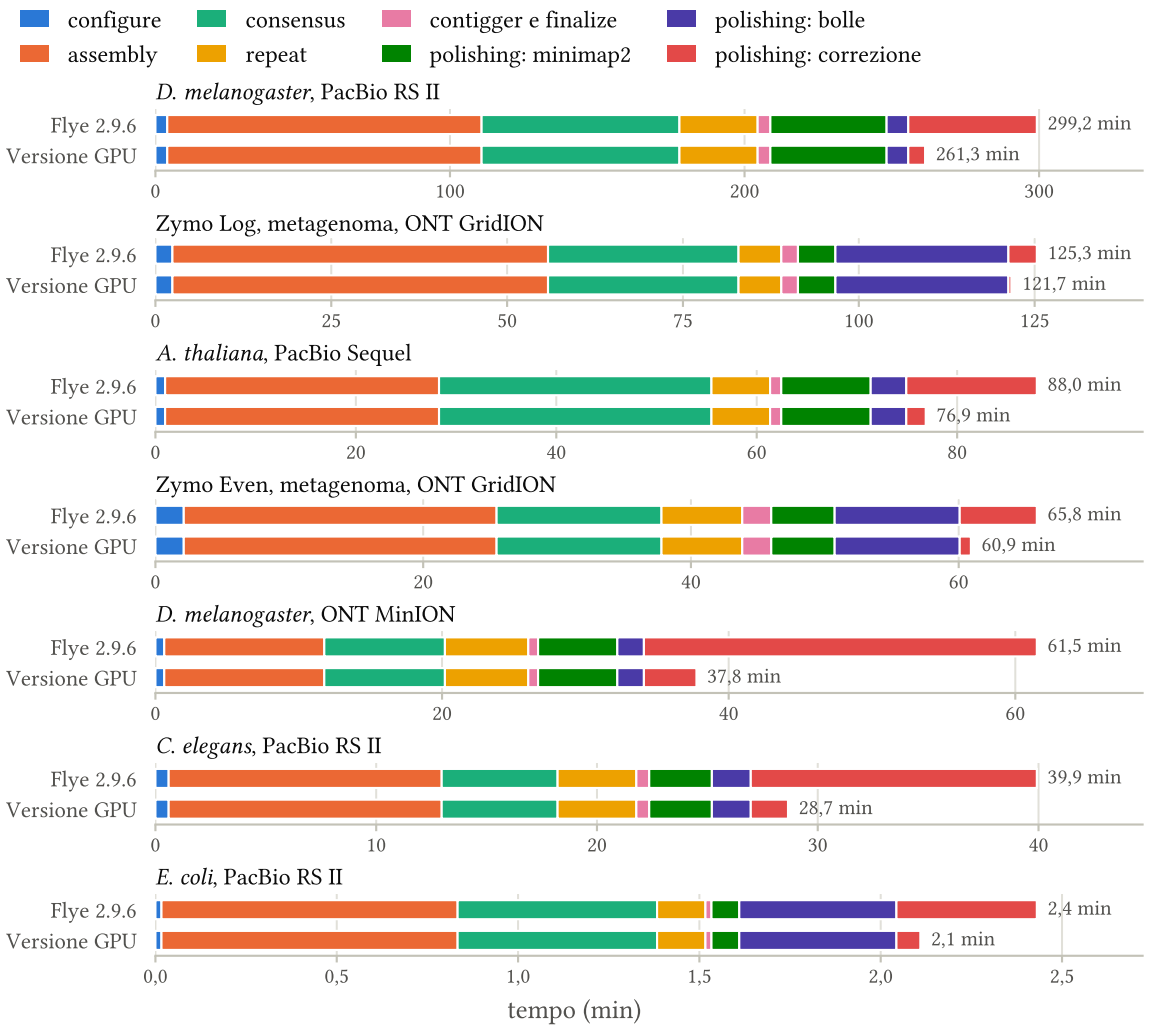


Figura 8: Tempo di ogni fase di Flye sui sette dataset, con Flye 2.9.6 e con la versione GPU, entrambe con 16 *thread*.

Capitolo 6

Conclusioni e sviluppi futuri

6.1 Rifinitura del software

La versione GPU assegna a ogni *thread worker* un *pool* di 32 bolle. Nella correzione di tutte le bolle di *C. elegans* il *pool* da 32 è generalmente il migliore e per questo è stato scelto.

La dimensione ottimale del *pool* diminuisce però all'aumentare dei *thread worker* che condividono la stessa GPU. Un possibile sviluppo del software potrebbe consistere nel parametrizzare la dimensione del *pool* in base al numero di *thread worker* ed eventualmente anche in base al numero di bolle da elaborare.

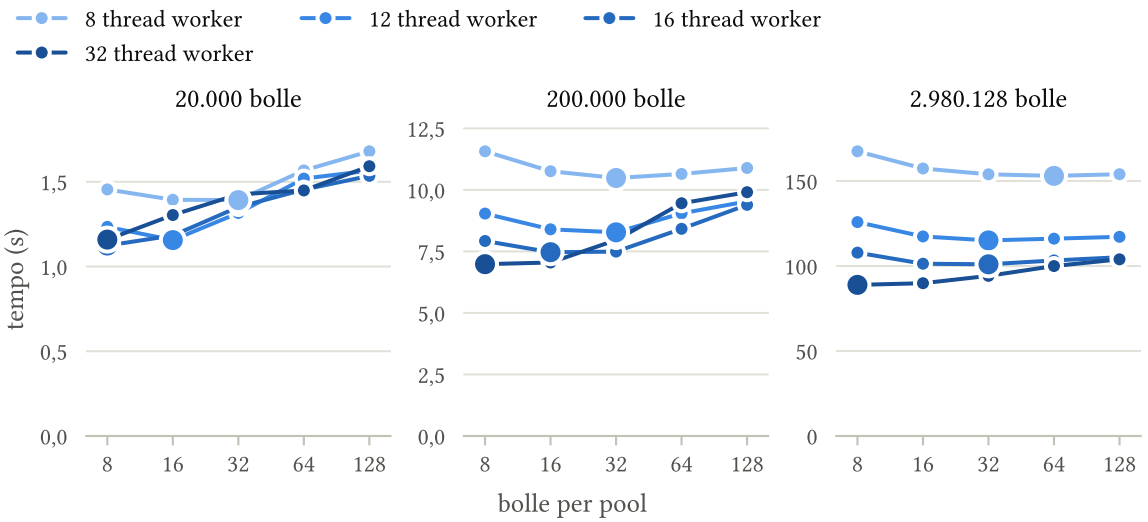


Figura 9: Correzione delle prime 20.000 e 200.000 bolle e di tutte le 2.980.128 bolle del dataset *C. elegans* con 8, 12, 16 e 32 *thread worker*, al variare del numero di bolle per *pool*. Il marcatore più grande di ogni linea indica il *pool* più veloce.

6.2 Una visione sull'intera *pipeline*

Nei miei test il tempo medio di calcolo della correzione delle bolle nella fase di *polishing* è passato dal 19,2% del tempo di calcolo totale a solamente il 3,6% (Figura 10).

Il passo successivo sarà applicare la stessa strategia di *porting* su GPU alle altre fasi di Flye, nello specifico accelerare la fase di *assembly* porterebbe a dei risultati significativi sull'intera *pipeline*.

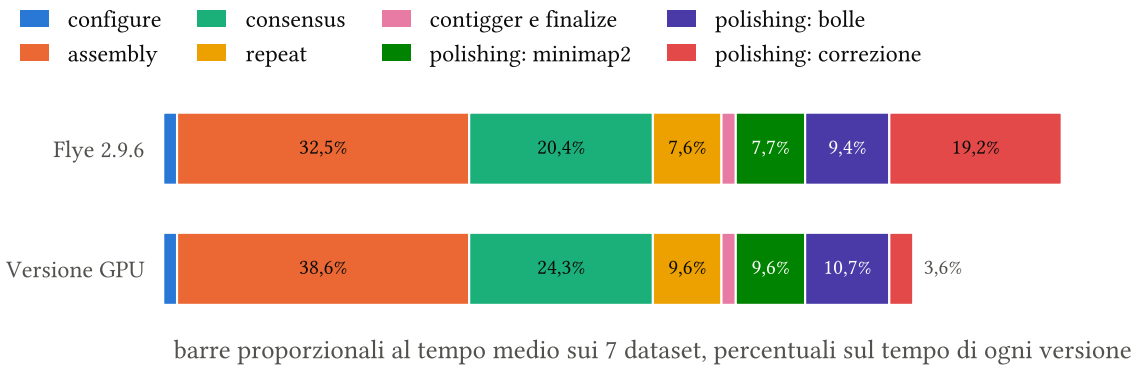


Figura 10: Tempo medio di ogni fase di Flye sui sette dataset della Figura 8, con Flye 2.9.6 e con la versione GPU. La lunghezza dei segmenti è proporzionale al tempo della fase rispetto al tempo complessivo di Flye 2.9.6, così le fasi non modificate hanno la stessa lunghezza nelle due barre; le percentuali indicano la quota del tempo complessivo della rispettiva versione.

Bibliografia

[1] Heng Li and Richard Durbin. Genome assembly in the telomere-to-telomere era. *Nature Reviews Genetics*, 25(9):658–670, 2024.

[2] Mikhail Kolmogorov, Jeffrey Yuan, Yu Lin, and Pavel A. Pevzner. Assembly of long, error-prone reads using repeat graphs. *Nature Biotechnology*, 37(5):540–546, 2019.

[3] Yu Lin, Jeffrey Yuan, Mikhail Kolmogorov, Max W. Shen, Mark Chaisson, and Pavel A. Pevzner. Assembly of long error-prone reads using de Bruijn graphs. *Proceedings of the National Academy of Sciences*, 113:E8396–E8405, 2016.

[4] NVIDIA Corporation. *CUDA Programming Guide*. Versione 13.4.2, `https://docs.nvidia.com/cuda/cuda-programming-guide/`.

[5] Mikhail Kolmogorov et al. Flye assembler. `https://github.com/fenderglass/Flye`. Versione 2.9.6.

[6] Heng Li. minimap2: A versatile pairwise aligner for genomic and spliced nucleotide sequences. `https://github.com/lh3/minimap2`.

[7] Saul B. Needleman and Christian D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3):443–453, 1970.



Lavoro di tesi sviluppato per il progetto di ricerca Abiosa